



UAC

Universidad Autónoma
de Campeche

"Del enigma sin albas a triángulos de luz"

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

FACULTAD DE INGENIERÍA

LIC. EN INGENIERÍA EN ENERGÍA

Unidad de aprendizaje: Seguridad Informática

Alumno(a): Inurreta Del Valle Rafael E

Nombre del trabajo: Practica 03_Algoritmo Híbrido

Fecha de entrega: 7/09/2026

Profesor@: Sergio A Noh Puch

Explicación del Funcionamiento del Algoritmo Híbrido

El cifrado híbrido combina las fortalezas de la criptografía simétrica (alta velocidad y eficiencia para procesar volúmenes grandes de datos) con la criptografía asimétrica (resolución segura del problema de intercambio de claves a través de canales inseguros). En esta práctica, se implementa un esquema híbrido compuesto por los algoritmos AES-256 (en modo CBC) y RSA (2048 bits con relleno OAEP).

Flujo de Cifrado (Generación y Envío - Función `get_Msj_And_Key`)

- **1. Generación de Clave Simétrica AES:** Se genera de forma aleatoria y segura una clave simétrica de 32 bytes (`AES_KEY_SIZE`) utilizando fuentes de entropía del sistema operativo (`os.urandom`).
- **2. Generación del Vector de Inicialización (IV):** Se genera un bloque de 16 bytes aleatorios (`AES_BLOCK_SIZE`) que actúa como Vector de Inicialización para el modo CBC, garantizando que cifrados idénticos produzcan textos cifrados completamente diferentes.
- **3. Cifrado del Mensaje con AES:** El mensaje en texto plano se ajusta mediante el esquema de relleno PKCS#7 para completar múltiplos exactos del bloque de 16 bytes. Posteriormente, se cifra utilizando AES-256 en modo CBC con la clave simétrica y el IV generados.
- **4. Cifrado Asimétrico de Credenciales (RSA-OAEP):** La clave simétrica AES y el IV se protegen cifrándolos de forma independiente utilizando la clave pública RSA del receptor bajo el esquema Optimal Asymmetric Encryption Padding (OAEP) con funciones hash SHA-256.
- **5. Empaquetado y Codificación:** Se devuelven los componentes cifrados convertidos a formato Base64 para facilitar su transmisión segura.

Flujo de Descifrado (Recepción - Función decifrar_mensaje)

- **1. Decodificación y Descifrado RSA:** El receptor toma los elementos cifrados en Base64, los decodifica y utiliza su clave privada RSA con relleno OAEP (SHA-256) para recuperar tanto el Vector de Inicialización (IV) como la clave secreta simétrica (AES).
- **2. Descifrado Simétrico AES:** Empleando la clave AES y el IV recuperados, se inicializa el descifrador AES-256-CBC para procesar el mensaje cifrado.
- **3. Eliminación de Relleno y Recuperación:** Se aplica el proceso inverso de PKCS#7 (unpadding) para retirar el relleno de bytes y obtener exactamente el mensaje original en texto plano.

2. Justificación de Elecciones de Diseño

Las decisiones arquitectónicas y criptográficas adoptadas en la implementación responden a rigurosos estándares actuales de seguridad de la información y eficiencia computacional:

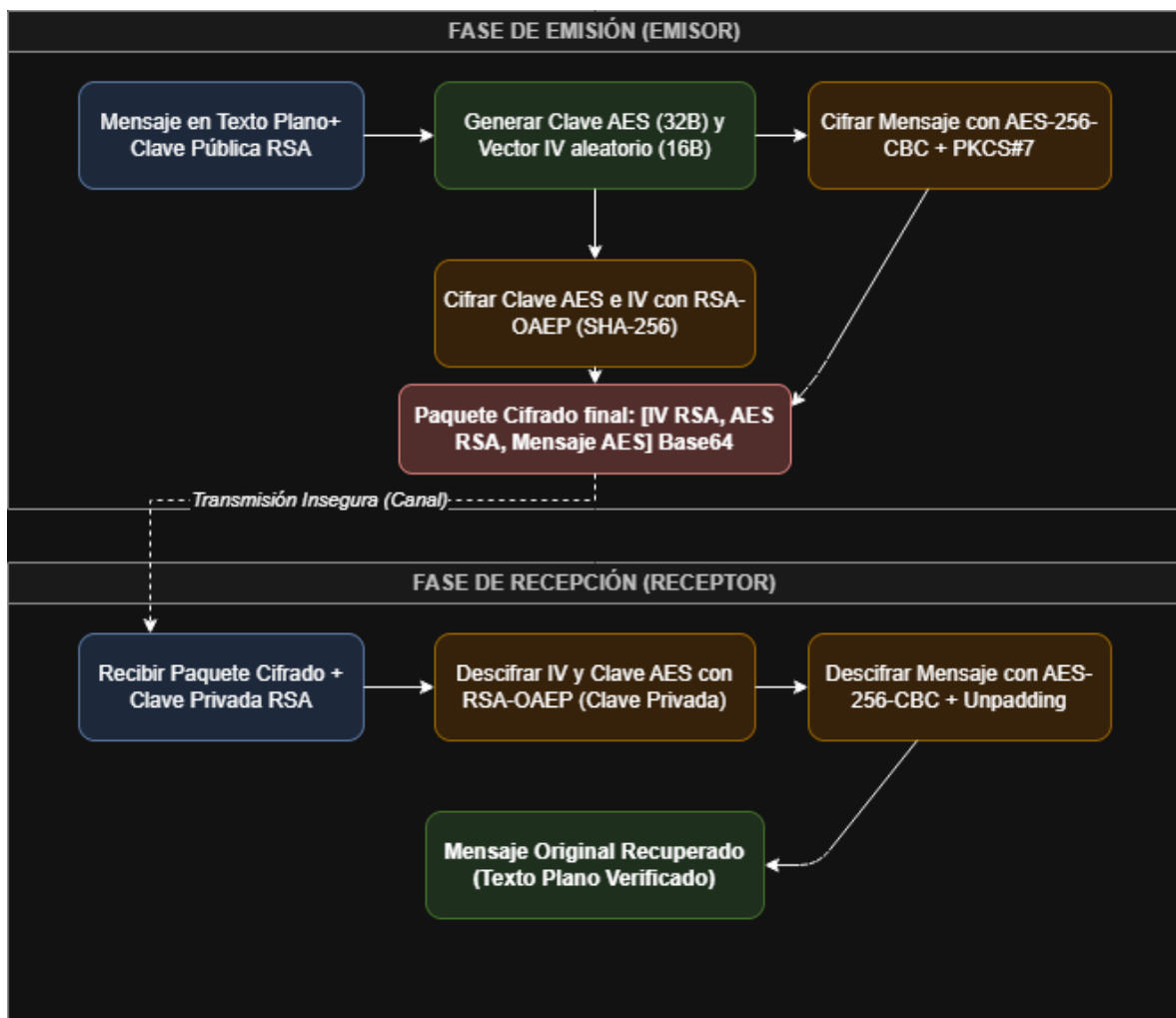
- **Tamaño de Clave RSA (2048 bits):** Proporciona un nivel de seguridad robusto frente a ataques de factorización actuales, equilibrando de forma óptima la seguridad a largo plazo y el costo computacional de las operaciones exponenciales modulares.
- **Esquema de Relleno RSA-OAEP:** Se selecciona OAEP frente a esquemas antiguos (como PKCS#1 v1.5) debido a su sólida seguridad demostrada contra ataques de texto cifrado adaptativo (ataques Bleichenbacher y de texto plano elegido). Además, se incorpora la función hash SHA-256.
- **Algoritmo Simétrico AES-256:** AES es el estándar de cifrado avanzado avalado por el NIST. Se utiliza con claves de 256 bits para garantizar la máxima resistencia teórica frente a ataques de fuerza bruta y criptoanálisis lineal/diferencial.
- **Modo de Operación CBC (Cipher Block Chaining):** Asegura que bloques idénticos de texto plano produzcan bloques de texto cifrado distintos al ser encadenados mediante operaciones XOR

con el bloque anterior y el IV aleatorio, previniendo la fuga de patrones estructurales.

- **Esquema de Relleno PKCS#7:** Indispensable en bloques simétricos para manejar mensajes de longitud arbitraria que no coinciden exactamente con los múltiplos de 16 bytes del bloque AES.

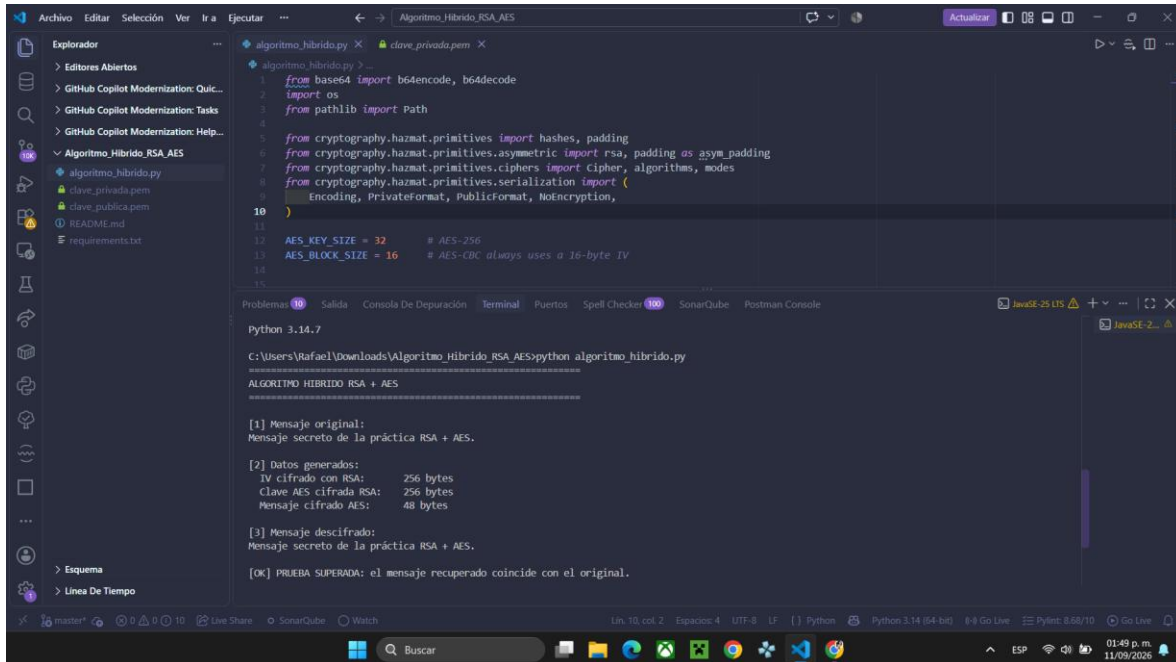
3. Diagrama de Flujo del Proceso

El esquema general detalla la interacción entre el emisor y el receptor en el sistema híbrido implementado:



4. Pruebas y Evidencias de Funcionamiento

A continuación, se presentan las capturas de pantalla y evidencias que demuestran la correcta ejecución del código fuente (algoritmo_hibrido.py), validando la generación de claves, el cifrado simétrico y asimétrico, y la exitosa recuperación del mensaje original.



```
from base64 import b64encode, b64decode
import os
from pathlib import Path

from cryptography.hazmat.primitives import hashes, padding
from cryptography.hazmat.primitives.asymmetric import rsa, padding as asym_padding
from cryptography.hazmat.primitives.ciphers import cipher, algorithms, modes
from cryptography.hazmat.primitives.serialization import (
    Encoding, PrivateFormat, PublicFormat, NoEncryption,
)

AES_KEY_SIZE = 32 # AES-256
AES_BLOCK_SIZE = 16 # AES-CBC always uses a 16-byte IV

Python 3.14.7

C:\Users\Rafael\Downloads\Algoritmo_Hibrido_RSA_AES>python algoritmo_hibrido.py

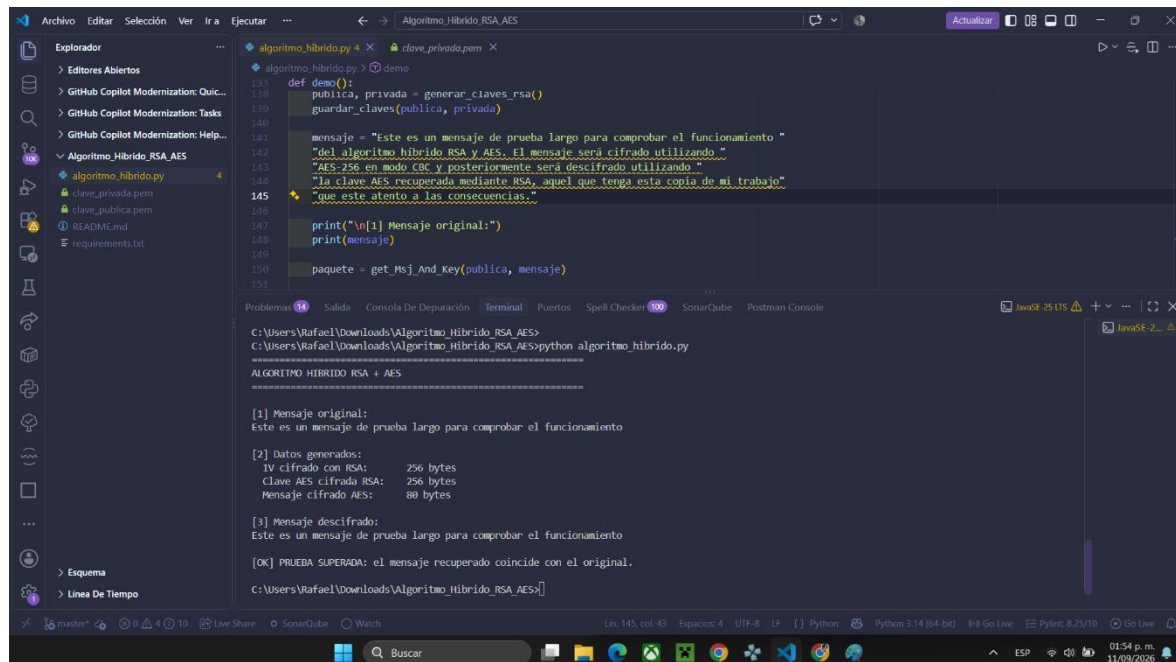
ALGORITMO HIBRIDO RSA + AES

[1] Mensaje original:
Mensaje secreto de la práctica RSA + AES.

[2] Datos generados:
IV cifrado con RSA: 256 bytes
Clave AES cifrada RSA: 256 bytes
Mensaje cifrado AES: 48 bytes

[3] Mensaje descifrado:
Mensaje secreto de la práctica RSA + AES.

[OK] PRUEBA SUPERADA: el mensaje recuperado coincide con el original.
```



```
def demo():
    publica, privada = generar_claves_rsa()
    guardar_claves(pública, privada)

    mensaje = "Este es un mensaje de prueba largo para comprobar el funcionamiento "
    "del algoritmo híbrido RSA y AES. El mensaje será cifrado utilizando "
    "AES-256 en modo CBC y posteriormente será descifrado utilizando "
    "la clave AES recuperada mediante RSA, aquel que tenga esta copia de mi trabajo "
    "que este atento a las consecuencias."

    print("\n[1] Mensaje original:")
    print(mensaje)

    paquete = get_Msj_And_Key(pública, mensaje)

Python 3.14.7

C:\Users\Rafael\Downloads\Algoritmo_Hibrido_RSA_AES>python algoritmo_hibrido.py

ALGORITMO HIBRIDO RSA + AES

[1] Mensaje original:
Este es un mensaje de prueba largo para comprobar el funcionamiento

[2] Datos generados:
IV cifrado con RSA: 256 bytes
Clave AES cifrada RSA: 256 bytes
Mensaje cifrado AES: 80 bytes

[3] Mensaje descifrado:
Este es un mensaje de prueba largo para comprobar el funcionamiento

[OK] PRUEBA SUPERADA: el mensaje recuperado coincide con el original.

C:\Users\Rafael\Downloads\Algoritmo_Hibrido_RSA_AES\]
```

```
def demo():
    publica, privada = generar_claves_rsa()
    guardar_claves(publica, privada)

    mensaje = "Segunda prueba: ¿éíóú ñ ¿?"

    print("\n[1] Mensaje original:")
    print(mensaje)

    paquete = get_Msj_And_Key(publica, mensaje)

    print("\n[2] Datos generados:")
    print(f"    IV cifrado con RSA: {len(b64decode(paquete['iv_cifrado_RSA']))} bytes")
    print(f"    Clave AES cifrada RSA: {len(b64decode(paquete['clave_AES_cifrada_RSA']))} bytes")
    print(f"    Mensaje cifrado AES: {len(b64decode(paquete['mensaje_cifrado_AES']))} bytes")

# Ejecución
demo()
```

Problemas 0 Salida Consola De Depuración Terminal Puertos Spell Checker 100 SonarQube Postman Console

```
C:\Users\Rafael\Downloads\Algoritmo_Hibrido_RSA_AES>python algoritmo_hibrido.py

ALGORITMO HIBRIDO RSA + AES

[1] Mensaje original:
Segunda prueba: ¿éíóú ñ ¿?

[2] Datos generados:
    IV cifrado con RSA: 256 bytes
    Clave AES cifrada RSA: 256 bytes
    Mensaje cifrado AES: 48 bytes

[3] Mensaje descifrado:
Segunda prueba: ¿éíóú ñ ¿?

[OK] PRUEBA SUPERADA: el mensaje recuperado coincide con el original.

C:\Users\Rafael\Downloads\Algoritmo_Hibrido_RSA_AES>
```

5. Análisis de Seguridad

El sistema híbrido implementado mitiga eficazmente las principales amenazas en la transmisión de información:

- **Confidencialidad Extrema:** El contenido del mensaje nunca viaja en texto plano y la clave simétrica está protegida por la robustez matemática de RSA-2048, inviable de factorizar con la tecnología actual.
- **Integridad y Autenticidad Estructural:** El uso de relleno OAEP previene ataques de relleno malicioso y asegura que cualquier alteración en el criptograma invalide el proceso de descifrado.
- **Independencia de Sesión:** Al generar una clave AES y un IV completamente aleatorios (`os.urandom`) para cada mensaje, se neutralizan ataques por análisis de tráfico repetitivo.